

LiMBS 1.0

Nangia Research Lab

User Manual

1. Overview

LiMBS1.0 is a line notation for describing lipid membrane simulation systems in a human-readable, machine-parseable format. It encodes membrane geometry, lipid composition, environmental conditions, and molecular structure in a single self-contained string.

LiMBS is resolution-independent: the same system header can describe an atomistic (AA), coarse-grained (CG), or super coarse-grained (SCG) membrane. The parser validates the notation and exports to JSON for downstream use with INSANE (planar membranes) or TS2CG (vesicles).

1.1 Supported Resolutions

Field	Description	Example
AA	Atomistic — heavy-atom SMILES notation	<code>rsln:AA</code>
CG	Coarse-grained — bead mapping notation	<code>rsln:CG</code>
SCG	Super coarse-grained — semantic group notation	<code>rsln:SCG</code>

1.2 Supported Geometries

Field	Description	Example
planar	Flat bilayer — use with INSANE	<code>type:planar</code>
vesicle	Spherical vesicle — use with TS2CG	<code>type:vesicle</code>

1.3 Dual-Block Architecture

Every LiMBS string consists of two blocks separated by the `|++|` delimiter:

```
[ System Block ]  
  
|++|  
  
[ Species Block ]
```

System Block — describes the membrane geometry, composition summary, box, solvent, and salt.

Species Block — defines the chemical structure of each species at the chosen resolution.

2. System Block

The system block is a sequence of fields separated by double-pipe delimiters (||). Every field except the final salt line must end with ||.

2.1 Field Reference

Field	Description	Example
LiMBS1.0	Version token — must be first	LiMBS1.0
rsln:	Resolution (AA, CG, or SCG)	rsln:CG
lsc:	Number of lipid species	lsc:2
uls:	Number of upper leaflet species	uls:2
lls:	Number of lower leaflet species	lls:1
{...}	Lipid metadata block	{DPPC:tails=16:0-16:0,...}
leaflet:	Leaflet composition (count or ratio)	leaflet:count{...}
type:	Membrane geometry	type:planar
box:	Box dimensions in nm	box:[10.0,10.0,8.0]nm
sol:	Solvent species name	sol:W
salt:	Salt concentration and ion type	salt:0.15M NaCl

2.2 Formatting Rules

CRITICAL

No spaces between field names and their values. Write rsln:CG not rsln: CG. This applies to every field without exception.

- Every header field except the final salt line must end with ||
- The lipid metadata block {...} must appear before the leaflet definition
- Metadata entries for multiple lipids are separated by semicolons inside one {...} block
- Do not split metadata into multiple {...} blocks

2.3 Complete Planar Example (CG)

```
LiMBS1.0 ||
rsln:CG ||
lsc:1 ||
uls:1 ||
lls:1 ||
{DPPC:tails=16:0-16:0,head=PC,charge=0.0} ||
```

```
leaflet:count{-u{DPPC:56}-l{DPPC:56}} ||  
type:planar ||  
box:[10.0,10.0,8.0]nm ||  
sol:W ||  
salt:0.15M NaCl
```

2.4 Complete Vesicle Example (CG)

```
LiMBS1.0 ||  
rsln:CG ||  
lsc:2 ||  
uls:2 ||  
lls:2 ||  
{POPC:tails=16:0-18:1,head=PC,charge=0.0;CHOL:tails=sterol,head=CHOL,charge=0.0} ||  
leaflet:ratio{-u{POPC:0.70,CHOL:0.30}-l{POPC:0.70,CHOL:0.30}} ||  
type:vesicle ||  
box:[20.0,20.0,20.0]nm ||  
sol:W ||  
salt:0.15M NaCl
```

NOTE

Vesicles always use `leaflet:ratio`. Planar membranes use `leaflet:count`. Using the wrong mode will cause a compatibility error in the exporter scripts.

3. Lipid Metadata

The metadata block encodes biological lipid identity independent of resolution or force field.

3.1 Format

```
{LIPID:tails=<tail_composition>,head=<headgroup>,charge=<net_charge>}
```

For multiple lipids, separate entries with semicolons inside a single block:

```
{DPPC:tails=16:0-16:0,head=PC,charge=0.0;POPE:tails=16:0-18:1,head=PE,charge=0.0}
```

3.2 Tail Composition Notation

Field	Description	Example
16:0	Palmitoyl — 16 carbons, saturated	tails=16:0
18:1	Oleoyl — 18 carbons, one double bond	tails=18:1
16:0-18:1	Two-chain lipid (e.g. POPC)	tails=16:0-18:1
sterol	Cholesterol ring structure	tails=sterol
mixed	Complex / heterogeneous (e.g. LPS)	tails=mixed

3.3 Common Lipid Metadata Examples

```
DPPC:tails=16:0-16:0,head=PC,charge=0.0  
POPC:tails=16:0-18:1,head=PC,charge=0.0  
POPE:tails=16:0-18:1,head=PE,charge=0.0  
POPG:tails=16:0-18:1,head=PG,charge=-1.0  
POPS:tails=16:0-18:1,head=PS,charge=-1.0  
POPA:tails=16:0-18:1,head=PA,charge=-1.0  
CHOL:tails=sterol,head=CHOL,charge=0.0  
LPS:tails=mixed,head=LPS,charge=-8.0  
PVCL2:tails=16:0-18:1-16:0-18:1,head=CL,charge=-2.0
```

NOTE

Metadata records chain length and unsaturation degree only. Exact double-bond positions are specified inside the species block.

4. Leaflet Composition

The leaflet field defines which lipids populate the upper and lower leaflets, along with their quantities or ratios.

4.1 Count Mode — Planar Membranes

```
leaflet:count{-u{DPPC:56}-l{DPPC:56}}
```

Mixed asymmetric example:

```
leaflet:count{  
-u{LPS:99}  
-l{PVCL2:26,PMPE:104,PMPG:13,PVPE:104,PVPG:26}  
}
```

4.2 Ratio Mode — Vesicles

```
leaflet:ratio{-u{POPC:0.50,CHOL:0.30,POPE:0.20}-l{POPC:0.50,CHOL:0.30,POPE:0.20}}
```

4.3 Bilayer Size Calculation (Planar)

```
leaflet_area    = box_x  x  box_y  
lipids_per_leaf = leaflet_area / APL  
  
# APL is per leaflet — do not divide by 2  
upper = round(lipids_per_leaf)  
lower = round(lipids_per_leaf)
```

5. Species Blocks

Species blocks define the chemical structure of each molecular species after the `| ++ |` separator, separated by `| + |`.

5.1 Ordering

- Lipid species first (order matches leaflet/metadata)
- Salt or ion species second
- Solvent species last

5.2 AA Species

```
DPPC=AA| [  
{ #T16=CCCCCCCCCCCCCCCC }  
  
[N] (C) (C) (C) CCOP(=O) (O) OCCOC(=O) [#T16] COC(=O) [#T16]  
]
```

5.3 CG Species — Two Layers

```
DPPC=CG| [  
  
[N(C) (C) (C) CC:^NC3] [OP(=O) (O) (O) :^PO4] [CCOC(=O) :^GL1] [COC(=O) :^GL2] [#TailA] [#TailB]  
  
{ #TailA=[>] [CCCC:^C1A] [CCCC:^C2A] [CCCC:^C3A] [CCCC:^C4A] }  
{ #TailB=[>] [CCCC:^C1B] [CCCC:^C2B] [CCCC:^C3B] [CCCC:^C4B] }  
  
|  
  
[^NC3] [^PO4] [^GL1] [^GL2] [#TailA] [#TailB]  
  
{ #TailA=[>] [^C1A] [^C2A] [^C3A] [^C4A] }  
{ #TailB=[>] [^C1B] [^C2B] [^C3B] [^C4B] }  
  
]
```

5.4 SCG Species — Three Layers

```
DPPC=SCG| [  
  
# Layer 1: chemical  
[N(C) (C) (C) CC:^NC3] [OP(=O) (O) (O) :^PO4] [CCOC(=O) :^GL1] [COC(=O) :^GL2] [#TailA] [#TailB]  
{ #TailA=[>] [CCCC:^C1A] [CCCC:^C2A] [CCCC:^C3A] [CCCC:^C4A] }  
{ #TailB=[>] [CCCC:^C1B] [CCCC:^C2B] [CCCC:^C3B] [CCCC:^C4B] }
```

```

|

# Layer 2: bead sequence
[^NC3][^PO4][^GL1][^GL2][#TailA][#TailB]
{#TailA=[>][^C1A][^C2A][^C3A][^C4A]}
{#TailB=[>][^C1B][^C2B][^C3B][^C4B]}

|

# Layer 3: semantic groups
[^^HEAD][^^GLYCEROL][^^TAIL]
{^^HEAD=[^NC3][^PO4]}
{^^GLYCEROL=[^GL1][^GL2]}
{^^TAIL=[#TailA][#TailB]}

]

```


6. Tail Fragment Naming

6.1 Default Convention

```
#T<carbons>_<double_bonds>
```

Examples:

#T16	16-carbon saturated
#T18_1	18-carbon, one double bond
#T18_2	18-carbon, two double bonds
#T20_4	20-carbon, four double bonds

6.2 With Double-Bond Position

```
#T<carbons>_<double_bonds>_D<position>
```

Examples:

#T18_1_D9	Oleoyl - delta-9 (POPC)
#T18_1_D11	Vaccenoyl - delta-11 (PVPE)
#T18_2_D9_D12	Linoleoyl

7. Solvent and Salt

7.1 Solvent Species

Field	Description	Example
sol:TIP3P	Atomistic water	W=AA [[O]]
sol:W	MARTINI CG water	W=CG [[OOOO:^W] [^W]]
sol:WW	SCG water	WW=SCG [[OOOOOOOO:^WW] [^WW] [^^SOLVENT]]

7.2 Salt Handling Rules

- Concentration > 0 → salt species block required
- Concentration = 0M → no salt species block
- Generate only the salt named in the header

CaCl₂ example (CG):

```
salt:0.15M CaCl2

CaCl2=CG | [
[Ca2+:^Ca] [Cl-:^Cl] [Cl-:^Cl]
|
[^Ca] [^Cl] [^Cl]
]
```

8. Validation Rules

8.1 Required Checks

- String must begin with LiMBS1.0
- Exactly one |++| separator
- No space between field key and value (rsln:CG not rsln: CG)
- Every leaflet species must have a species block
- Every referenced fragment must be defined
- Species names must be unique and consistent (CHOL not Chol)
- Resolution must match all species block tags
- Solvent must be defined as a species block
- Salt > 0M requires a salt species block
- All brackets must be balanced
- Metadata block must appear before leaflet field

8.2 Common Errors and Fixes

Field	Description	Example
Space after key	rsln: CG instead of rsln:CG	Remove the space
Missing	Header field not ending with	Add to the field
Missing ++	Species blocks not separated	Add ++ between blocks
Wrong leaflet mode	count for vesicle or ratio for planar	ratio for vesicle, count for planar
Undefined species	Leaflet lists lipid with no species block	Add the species block
Salt with 0M	NaCl block present when salt:0M NaCl	Remove the salt block
Split metadata	Two separate {...} blocks	Merge into one block

8.3 Canonical Ordering

- Lipid names sorted alphabetically in metadata and leaflet
- Upper leaflet before lower leaflet
- Species blocks sorted alphabetically after |++|
- All units explicit (nm for box, M for salt)

9. Full Examples

9.1 DPPC Planar Bilayer — Atomistic

```
LiMBS1.0 ||
rsl:n:AA ||
lsc:1 ||
uls:1 ||
lls:1 ||
{DPPC:tails=16:0-16:0,head=PC,charge=0.0} ||
leaflet:count{-u{DPPC:158}-l{DPPC:158}} ||
type:planar ||
box:[10.0,10.0,8.0]nm ||
sol:TIP3P ||
salt:0.15M NaCl

|++|

DPPC=AA| [
{#T16=CCCCCCCCCCCCCCCC}

[N] (C) (C) (C) CCOP (=O) (O) OCCOC (=O) [#T16] COC (=O) [#T16]
]

|+|

NaCl=AA| [
[Na+] [Cl-]
]

|+|

TIP3P=AA| [
[O]
]
```

9.2 DOPC/POPA/POPE Planar Bilayer — CG

```
LiMBS1.0 ||
rsl:n:CG ||
lsc:3 ||
uls:3 ||
lls:3 ||
{DOPC:tails=18:1-18:1,head=PC,charge=0.0;
 POPA:tails=16:0-18:1,head=PA,charge=-1.0;
 POPE:tails=16:0-18:1,head=PE,charge=0.0} ||
leaflet:count{-u{DOPC:40,POPA:10,POPE:10}-l{DOPC:40,POPA:10,POPE:10}} ||
type:planar ||
box:[12.0,12.0,9.0]nm ||
sol:W ||
salt:0.15M NaCl
```

9.3 POPC/CHOL Vesicle — CG

```

LiMBS1.0 ||
rsln:CG ||
lsc:2 ||
uls:2 ||
lls:2 ||
{CHOL:tails=sterol,head=CHOL,charge=0.0;
 POPC:tails=16:0-18:1,head=PC,charge=0.0} ||
leaflet:ratio{-u{CHOL:0.30,POPC:0.70}-l{CHOL:0.30,POPC:0.70}} ||
type:vesicle ||
box:[20.0,20.0,20.0]nm ||
sol:W ||
salt:0.15M NaCl

```

9.4 E. coli Inner Membrane — CG (Asymmetric)

```

LiMBS1.0 ||
rsln:CG ||
lsc:5 ||
uls:1 ||
lls:4 ||
{LPS:tails=mixed,head=K12_core_LPS,charge=-8.0;
 PVCL2:tails=16:0-18:1-16:0-18:1,head=CL,charge=-2.0;
 PMPE:tails=14:0-16:1,head=PE,charge=0.0;
 PMPG:tails=14:0-16:1,head=PG,charge=-1.0;
 PVPE:tails=16:0-18:1,head=PE,charge=0.0} ||
leaflet:count{
 -u{LPS:99}
 -l{PVCL2:26,PMPE:104,PMPG:13,PVPE:104}
} ||
type:planar ||
box:[20.0,20.0,12.0]nm ||
sol:W ||
salt:0.15M NaCl

```

10. Quick Reference

Delimiters

Field	Description	Example
<code> </code>	Separates system header fields	<code>rsln:CG </code>
<code>+++ </code>	Separates system block from species block	<code>system +++ species</code>
<code> + </code>	Separates individual species blocks	<code>DPPC=CG [...] + W=CG [...]</code>
<code> </code>	Separates layers within a species block	<code>layer1 layer2 layer3</code>

Geometry vs Leaflet Mode

Field	Description	Example
<code>type:planar</code>	Planar bilayer	<code>leaflet:count</code> – use with INSANE
<code>type:vesicle</code>	Spherical vesicle	<code>leaflet:ratio</code> – use with TS2CG

Resolution Layers

Field	Description	Example
<code>AA</code>	1 layer	SMILES structure
<code>CG</code>	2 layers	chemical mapping bead sequence
<code>SCG</code>	3 layers	chemical bead sequence semantic groups

Most Common Mistakes

- Space after field key: `rsln: CG` instead of `rsln:CG`
- Missing `||` on a header field
- Using `leaflet:count` for a vesicle (should be `leaflet:ratio`)
- Lipid name mismatch between metadata and species block
- Salt block present when concentration is 0M

- Forgetting $|+|$ between species blocks

11. Using the Parser Scripts

The LiMBS repository includes three Python scripts. All require Python 3.6 or later and no additional pip packages. Place your LiMBS notation in a plain text file (e.g. system.txt) before running any script.

Field	Description	Example
LiMBSv1_parser.py	Core parser — validation, summary, JSON export, canonicalization	<code>python LiMBSv1_parser.py system.txt</code>
LiMBS_insane.py	Planar membrane builder — converts CG planar LiMBS to INSANE input and runs INSANE	<code>python LiMBS_insane.py system.txt</code>
LiMBS_vesicle.py	Vesicle builder — converts CG vesicle LiMBS to TS2CG input.str and runs TS2CG	<code>python LiMBS_vesicle.py system.txt</code>

11.1 LiMBSv1_parser.py — Core Parser

This is the foundation script. It reads a LiMBS text file, validates it against all syntax rules, and can output a human-readable summary, a canonical form, or a JSON export.

Command-line flags

Field	Description	Example
(none)	Validate and print a parsed summary to the terminal	<code>python LiMBSv1_parser.py system.txt</code>
--validate-only	Validate only — exit 0 on pass, exit 1 on failure	<code>python LiMBSv1_parser.py system.txt --validate-only</code>
--canonical	Print the canonical LiMBS string to stdout	<code>python LiMBSv1_parser.py system.txt --canonical</code>
--json	Print the full parsed system as JSON to stdout	<code>python LiMBSv1_parser.py system.txt --json</code>
--quiet	Suppress the summary — only print pass/fail	<code>python LiMBSv1_parser.py system.txt --quiet</code>

Validation

Run without any flags to validate and see a full parsed summary:

```
python LiMBSv1_parser.py system.txt
```


On a valid file the terminal output will look like this:

```
LiMBS1.0 parsed system
Version      : 1.0
Resolution   : CG
Lipid sp. count : 1
U leaflet species: 1
L leaflet species: 1
Type         : planar
Box          : [10.0, 10.0, 8.0] nm
Solvent      : W
Salt         : 0.15 M NaCl
Leaflet mode : count
Upper leaflet : {'DPPC': 56}
Lower leaflet : {'DPPC': 56}
Lipid metadata : ['DPPC']
Species defs  : ['DPPC', 'NaCl', 'W']
DPPC         : CG, 2 layer(s)
  Fragments defined: ['TailA', 'TailB']
  Fragments used   : ['TailA', 'TailB']

Validation passed.
```

Use `--validate-only` for scripting or CI pipelines — it prints only pass/fail and sets the exit code:

```
python LiMBSv1_parser.py system.txt --validate-only
# exit code 0 = passed, exit code 1 = failed
```

On failure, an error code and message are printed:

```
LMB-E002: LiMBS string must contain exactly one |++| separator
# or
LMB-E010: Species 'DPPC' in upper leaflet has no species block defined.
```

Canonicalization

The `--canonical` flag outputs a normalized, alphabetically sorted version of a valid LiMBS string. This is useful for comparing two systems that describe the same membrane, or for storing a standardized representation.

```
python LiMBSv1_parser.py system.txt --canonical
```

To save the canonical form to a file:

```
python LiMBSv1_parser.py system.txt --canonical > system_canonical.txt
```

NOTE

Canonicalization only succeeds on a valid file. If the file fails validation, the script exits with an error before producing canonical output.

JSON Export

The `--json` flag outputs the full parsed system as a structured JSON object. This is the format consumed internally by `LiMBS_insane.py` and `LiMBS-vesicle.py`.

```
python LiMBSv1_parser.py system.txt --json
```

To save the JSON to a file:

```
python LiMBSv1_parser.py system.txt --json > system.json
```

The JSON output includes the resolution, membrane type, box, solvent, salt, leaflet composition, lipid metadata, species block details, and any warnings or errors.

11.2 LiMBS_insane.py — Planar Membrane Builder

Converts a validated CG planar LiMBS file into an INSANE command and optionally executes it to produce `membrane.gro` and `membrane.top`. Requires `LiMBSv1_parser.py` and `insane.py` in the same directory.

CRITICAL

Only works with CG resolution and type:planar. Using AA, SCG, or type:vesicle will raise a compatibility error before any file is written.

Command-line flags

Field	Description	Example
input	LiMBS .txt file (required positional argument)	<code>system.txt</code>
--dry-run	Print the INSANE command without executing it	<code>--dry-run</code>
--write-json PATH	Save the intermediate parsed JSON to a file	<code>--write-json out.json</code>
--from-json	Start from a previously saved LiMBS JSON file	<code>--from-json out.json</code>
--insane-script	Path to <code>insane.py</code> (default: <code>insane.py</code>)	<code>--insane-script insane.py</code>
--python-exe	Python executable to run INSANE (default: <code>python3</code>)	<code>--python-exe python</code>
--out-gro	Output .gro filename (default: <code>membrane.gro</code>)	<code>--out-gro bilayer.gro</code>

--out-top	Output .top filename (default: membrane.top)	--out-top bilayer.top
--rand	INSANE randomisation seed (default: 0.1)	--rand 0.5
--solr	INSANE solute rotational freedom (default: 0.5)	--solr 0.3

Typical workflow

Step 1 — validate your file first:

```
python LiMBSv1_parser.py system_CG.txt --validate-only
```

Step 2 — dry-run to inspect the generated INSANE command:

```
python LiMBS_insane.py system_CG.txt --dry-run
```

Step 3 — run INSANE to produce the membrane files:

```
python LiMBS_insane.py system_CG.txt --insane-script insane.py
```

Expected terminal output on success:

```
Parsing LiMBS file: system_CG.txt
  Validation: PASSED
  INSANE compatibility: OK

Generated INSANE command:
  python3 insane.py -u DPPC:56 -l DPPC:56 -x 10.0 -y 10.0 -z 8.0 -sol W -salt 0.15 ...

Running INSANE...

INSANE completed successfully.
  Coordinates : membrane.gro
  Topology    : membrane.top
```

OUTPUT

membrane.gro — GROMACS coordinate file ready for solvation and energy minimization
 membrane.top — GROMACS topology file listing lipid and solvent molecule counts

11.3 LiMBS_vesicle.py — Vesicle Builder

Converts a validated CG vesicle LiMBS file into a TS2CG input.str, then runs TS2CG PLM and PCG steps to build the vesicle. Requires LiMBSv1_parser.py, a .tsi geometry file, and a Martini lipid library.

CRITICAL

Only works with CG resolution and type:vesicle. The TS2CG executable must be in your PATH.

Command-line flags

Field	Description	Example
input	LiMBS .txt file (required positional argument)	vesicle.txt
--validate-only	Parse and validate only — write nothing	--validate-only
--write-str-only	Write input.str only — do not run TS2CG	--write-str-only
--dry-run	Print TS2CG commands without executing them	--dry-run
--tsi FILE	TS2CG geometry file (default: sphere.tsi)	--tsi mySphere.tsi
--lib FILE	Martini lipid library (default: Martini3.LIB)	--lib Martini3.LIB
--str-file FILE	TS2CG composition file (default: input.str)	--str-file myinput.str
--out-name NAME	Output prefix (default: vesicle)	--out-name myvesicle
--bilayer-thickness	PLM bilayer thickness in nm (default: 3.0)	--bilayer-thickness 3.8
--rescale X Y Z	PLM rescale factors (default: 4.8 4.8 4.8)	--rescale 5.0 5.0 5.0
--mashno INT	PLM Mashno value (default: 4)	--mashno 6
--bond-length	PCG bond length in nm (default: 0.2)	--bond-length 0.25

Typical workflow

Step 1 — validate your vesicle file:

```
python LiMBSv1_parser.py vesicle.CG.txt --validate-only
```

Step 2 — write input.str only (inspect before running TS2CG):

```
python LiMBS_vesicle.py vesicle_CG.txt --write-str-only
cat input.str
```

The generated input.str will look like:

```
[Lipids List]
Domain 0
POPC 1 1 0.7
CHOL 1 1 0.3
POPC 2 1 0.7
CHOL 2 1 0.3
End
```

Step 3 — dry-run to inspect TS2CG commands:

```
python LiMBS_vesicle.py vesicle_CG.txt --dry-run
```

Step 4 — full run:

```
python LiMBS_vesicle.py vesicle_CG.txt --tsi sphere.tsi --lib Martini3.LIB
```

Expected terminal output on success:

```
Parsing LiMBS file: vesicle_CG.txt
  Validation: PASSED

Parsed LiMBS vesicle system:
  Resolution      : CG
  Geometry        : vesicle
  Box             : [20.0, 20.0, 20.0] nm
  Solvent         : W
  Salt            : 0.15 M NaCl
  Outer (upper)   : {'POPC': 0.7, 'CHOL': 0.3}
  Inner (lower)   : {'POPC': 0.7, 'CHOL': 0.3}

  TS2CG compatibility: OK

[TS2CG input.str]
  Wrote: input.str

[Step 1] TS2CG PLM
  TS2CG PLM -TSfile sphere.tsi -bilayerThickness 3.0 ...

[Step 2] TS2CG PCG
  TS2CG PCG -dts point -str input.str -Bondlength 0.2 ...

TS2CG completed successfully.
  Output prefix : vesicle
```

OUTPUT

vesicle.gro — GROMACS coordinate file of the assembled vesicle vesicle.top —
GROMACS topology file input.str — TS2CG composition file (kept for reference)

11.4 Recommended Workflow

The recommended order for any new LiMBS system is:

1. Write the LiMBS notation in a text file
2. Validate with `LiMBSv1_parser.py` (no flags) — read the summary carefully
3. Optionally save canonical form with `--canonical` — use as the archived version
4. Optionally export JSON with `--json` — inspect or pass to other tools
5. Dry-run the exporter script — verify the generated command before execution
6. Run the exporter to produce `.gro` and `.top` files

NOTE

Always validate with `LiMBSv1_parser.py` before running `LiMBS_insane.py` or `LiMBS_vesicle.py`. The exporter scripts also validate internally, but running the parser first gives a clearer error message if something is wrong.